

```
"""
```

```
tryon_engine.py
```

```
Core logic for a *non-generative* virtual  
saree try-on.
```

```
No text prompts, no diffusion / GAN image  
generation of any kind.
```

```
The result image is produced entirely by  
classical computer vision:
```

1. Pose estimation on the customer
photo (MediaPipe PoseLandmarker)
 -> locate shoulders, hips, ankles.
2. Body segmentation, produced by the
SAME PoseLandmarker call
 (output_segmentation_masks=True) ->
know which pixels are
 "person".
3. The saree photo is treated as a flat
source image and is
 geometrically WARPED (perspective
transform) onto the torso
 and leg regions defined by the pose
landmarks.
4. The warped saree is composited onto
the customer photo with
 a feathered alpha mask, restricted
to the segmented body area.

This is the same family of technique used by classic "image-based virtual try-on" systems (warp-and-blend pipelines), as opposed to prompt-driven generative models. Because it is geometric warping of the ACTUAL uploaded saree photo (not a synthesized one), the output always shows the real saree's actual print/colour/texture.

Model files

MediaPipe's Tasks API loads its model from a local ``.task`` file instead of bundling it in the pip package. Run ``python download_models.py`` once (requires internet) before starting the API – see README.md.

Limitations (be upfront with users):

- It approximates drape; it does not physically simulate cloth folds.
- Works best with a front-facing, mostly-upright, single-person photo.
- The saree source image should show the saree reasonably flat/clear (e.g. laid out, on a mannequin, or a clean product shot).

"""

```
from __future__ import annotations
```

```
import io
```

```
import os
from dataclasses import dataclass

import cv2
import numpy as np
from PIL import Image

import mediapipe as mp
from mediapipe.tasks.python import vision
as mp_vision
from mediapipe.tasks.python import core as
mp_core

MODEL_DIR =
os.path.join(os.path.dirname(os.path.abspath(__file__)), "models")
POSE_MODEL_PATH = os.path.join(MODEL_DIR,
"pose_landmarker_full.task")

class NoPersonDetectedError(Exception):
    pass

class ModelNotFoundError(Exception):
    pass

@dataclass
class PoseLandmarks:
    left_shoulder: tuple
    right_shoulder: tuple
    left_hip: tuple
    right_hip: tuple
```

```

        left_ankle: tuple
        right_ankle: tuple

_detector = None # lazily created, cached
PoseLandmarker instance

def _get_detector() ->
mp_vision.PoseLandmarker:
    global _detector
    if _detector is not None:
        return _detector

    if not os.path.exists(POSE_MODEL_PATH):
        raise ModelNotFoundError(
            f"Pose model not found at
{POSE_MODEL_PATH}. "
            f"Run `python
download_models.py` once (needs internet)
to fetch it."
        )

    base_options =
mp_core.BaseOptions(model_asset_path=POSE_M
ODEL_PATH)
    options =
mp_vision.PoseLandmarkerOptions(
        base_options=base_options,

running_mode=mp_vision.RunningMode.IMAGE,
        num_poses=1,
        min_pose_detection_confidence=0.5,
        min_pose_presence_confidence=0.5,

```

```

        min_tracking_confidence=0.5,
        output_segmentation_masks=True,
    )
    _detector =
mp_vision.PoseLandmarker.create_from_options(options)
    return _detector

# MediaPipe Pose landmark indices
# (BlazePose 33-point topology)
_LEFT_SHOULDER, _RIGHT_SHOULDER = 11, 12
_LEFT_HIP, _RIGHT_HIP = 23, 24
_LEFT_ANKLE, _RIGHT_ANKLE = 27, 28

def _run_pose(image_rgb: np.ndarray):
    """Runs PoseLandmarker once; returns
    (PoseLandmarks, mask[H,W] float32)."""
    h, w = image_rgb.shape[:2]
    mp_image =
mp.Image(image_format=mp.ImageFormat.SRGB,
data=image_rgb)

    detector = _get_detector()
    result = detector.detect(mp_image)

    if not result.pose_landmarks:
        raise NoPersonDetectedError(
            "No person / body pose could be
detected in the customer photo."
        )

    lm = result.pose_landmarks[0] # first

```

detected person

```
def pt(i):
    return (lm[i].x * w, lm[i].y * h)

landmarks = PoseLandmarks(
    left_shoulder=pt(_LEFT_SHOULDER),
    right_shoulder=pt(_RIGHT_SHOULDER),
    left_hip=pt(_LEFT_HIP),
    right_hip=pt(_RIGHT_HIP),
    left_ankle=pt(_LEFT_ANKLE),
    right_ankle=pt(_RIGHT_ANKLE),
)

if result.segmentation_masks:
    mask =
result.segmentation_masks[0].numpy_view().a
stype(np.float32)
else:
    mask = np.ones((h, w),
dtype=np.float32)

return landmarks, mask

def _load_saree_rgba(saree_bytes: bytes) ->
np.ndarray:
    img =
Image.open(io.BytesIO(saree_bytes)).convert
("RGBA")
    return np.array(img)

def _warp_region(src_rgba: np.ndarray,
```

```

src_quad: np.ndarray,
            dst_quad: np.ndarray,
out_w: int, out_h: int) -> np.ndarray:
    M =
cv2.getPerspectiveTransform(src_quad.astype
(np.float32),

dst_quad.astype(np.float32))
    warped = cv2.warpPerspective(
        src_rgba, M, (out_w, out_h),
        flags=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_CONSTANT,
        borderValue=(0, 0, 0, 0),
    )
    return warped


def _expand_point(p, center, factor):
    """Push point p away from center by
    `factor` (>1 expands outward)."""
    return (center[0] + (p[0] - center[0])
    * factor,
            center[1] + (p[1] - center[1])
    * factor)


def _feather_mask(mask_u8: np.ndarray,
blur: int = 15) -> np.ndarray:
    blur = blur if blur % 2 == 1 else blur
    + 1
    return cv2.GaussianBlur(mask_u8, (blur,
blur), 0)

```

```

def virtual_tryon(customer_bytes: bytes,
saree_bytes: bytes,
                    drape_width_factor:
float = 1.18,
                    feather: int = 21) ->
bytes:
    """
        Main entry point.

        Parameters
        -----
        customer_bytes : raw bytes of the
customer's photo (jpg/png)
        saree_bytes    : raw bytes of the saree
photo (jpg/png, ideally
                        showing the saree laid
flat / on a mannequin)
        drape_width_factor : how much wider
than the shoulder/hip span the
                        draped cloth should
render (>1.0 widens it, since
                        a real saree drape
extends a bit beyond the body).
        feather : blur radius (px) used to
soften the composite edge.

        Returns
        -----
        PNG image bytes of the customer photo
with the saree warped onto it.
    """
    customer_pil =
Image.open(io.BytesIO(customer_bytes)).conv
ert("RGB")

```

```

customer_rgb = np.array(customer_pil)
h, w = customer_rgb.shape[:2]

pose, person_mask =
_run_pose(customer_rgb)

saree_rgba =
_load_saree_rgba(saree_bytes)
sh, sw = saree_rgba.shape[:2]

src_upper = np.array([[0, 0], [sw, 0],
[sw, sh * 0.55], [0, sh * 0.55]])
src_lower = np.array([[0, sh * 0.40],
[sw, sh * 0.40], [sw, sh], [0, sh]])

mid_shoulder = ((pose.left_shoulder[0]
+ pose.right_shoulder[0]) / 2,
                (pose.left_shoulder[1]
+ pose.right_shoulder[1]) / 2)
mid_hip = ((pose.left_hip[0] +
pose.right_hip[0]) / 2,
           (pose.left_hip[1] +
pose.right_hip[1]) / 2)

ls = _expand_point(pose.left_shoulder,
mid_shoulder, drape_width_factor)
rs = _expand_point(pose.right_shoulder,
mid_shoulder, drape_width_factor)
lh = _expand_point(pose.left_hip,
mid_hip, drape_width_factor)
rh = _expand_point(pose.right_hip,
mid_hip, drape_width_factor)
la = _expand_point(pose.left_ankle,
mid_hip, drape_width_factor * 0.9)

```

```

        ra = _expand_point(pose.right_ankle,
mid_hip, drape_width_factor * 0.9)

        neck_y = mid_shoulder[1] - (mid_hip[1]
- mid_shoulder[1]) * 0.12
        ls_top = (ls[0], neck_y)
        rs_top = (rs[0], neck_y)
        hem_y = max(la[1], ra[1]) + (max(la[1],
ra[1]) - mid_hip[1]) * 0.05
        la_bot = (la[0], hem_y)
        ra_bot = (ra[0], hem_y)

        dst_upper = np.array([ls_top, rs_top,
rh, lh])
        dst_lower = np.array([lh, rh, ra_bot,
la_bot])

        warped_upper = _warp_region(saree_rgba,
src_upper, dst_upper, w, h)
        warped_lower = _warp_region(saree_rgba,
src_lower, dst_lower, w, h)

        garment = warped_lower.copy()
        upper_alpha = warped_upper[:, :,
3:4].astype(np.float32) / 255.0
        garment[:, :, :3] = (warped_upper[:, :,
:3] * upper_alpha +
                                garment[:, :, :3]
* (1 - upper_alpha)).astype(np.uint8)
        garment[:, :, 3] =
np.maximum(warped_upper[:, :, 3],
garment[:, :, 3])

        person_mask_u8 = (np.clip(person_mask,

```

```

0, 1) * 255).astype(np.uint8)
    person_mask_u8 =
cv2.dilate(person_mask_u8, np.ones((9, 9),
np.uint8))

    garment_alpha = garment[:, :,
3].astype(np.float32) / 255.0
    combined_alpha = garment_alpha *
(person_mask_u8.astype(np.float32) / 255.0)
    combined_alpha_u8 = (combined_alpha *
255).astype(np.uint8)
    combined_alpha_u8 =
_feather_mask(combined_alpha_u8, feather)
    alpha =
(combined_alpha_u8.astype(np.float32) /
255.0)[ :, :, None]

    customer_f =
customer_rgb.astype(np.float32)
    garment_rgb = garment[:, :,
:3].astype(np.float32)

    result_rgb = customer_f * (1 - alpha) +
garment_rgb * alpha
    result_rgb = np.clip(result_rgb, 0,
255).astype(np.uint8)

    out_img = Image.fromarray(result_rgb,
mode="RGB")
    buf = io.BytesIO()
    out_img.save(buf, format="PNG")
    return buf.getvalue()

```

